

Conjuntos concurrentes:
con y sin locks

Comportamiento de objetos concurrentes

El comportamiento de los objetos concurrentes se describe mediante:

Comportamiento de objetos concurrentes

El comportamiento de los objetos concurrentes se describe mediante:

- **Correctitud (Safety):** Algún tipo de **equivalencia con el comportamiento secuencial**.
 - **Consistencia Secuencial** Condición fuerte.
 - **Linealizabilidad (Linearizability)** Aún más fuerte. Soporta **composición**.
 - **Consistencia en Reposo (Quiescent)** Consistencia débil.

Comportamiento de objetos concurrentes

El comportamiento de los objetos concurrentes se describe mediante:

- **Progreso (Liveness)**
 - **Bloqueante (blocking)** La demora de un proceso puede impedir que los demás progresen.
 - **No bloqueante (nonblocking)** La demora de un proceso no puede demorar a los demás indefinidamente.

Objeto como TDA

Un contenedor de datos + un conjunto de operaciones (métodos), que proveen la única vía para manipular los datos.

- **Clase o definición de tipo:** Cada objeto tiene una clase que define:
 - estado
 - comportamiento (operaciones)

Especificación de operaciones

- **Precondición:** Describe los estados en que puede invocarse una operación.
- **Postcondición:** Describe el valor de retorno de la operación y el estado del objeto *después* de ejecutar la operación.

Ejemplo: Especificación secuencial de cola FIFO

Estados de una cola

Una secuencia de elementos, posiblemente vacía.

- **Método $\text{enq}(z)$:**
 - *Precondición:* True.
 - *Postcondición:* El estado de la cola pasa a $q \cdot z$ (con q el estado al inicio).
- **Método $\text{deq}()$:**
 - *Precondición:* True.
 - *Postcondición:*
 - Si q vacía, lanza `EmptyException` y no cambia estado.
 - Si $q = a \cdot q'$, retorna a y cambia estado a q' .

En el modelo secuencial

Un **único proceso** manipula a los objetos.

- La especificación describe el estado antes y después de cada operación.
- se ignoran los estados intermedios durante la ejecución del método.
- Los estados relevantes son **entre llamadas** a métodos.

Especificación Secuencial

En el modelo secuencial

Un **único proceso** manipula a los objetos.

- La especificación describe el estado antes y después de cada operación.
- se ignoran los estados intermedios durante la ejecución del método.
- Los estados relevantes son **entre llamadas** a métodos.

Falla en Concurrencia

- Las ejecuciones de operaciones se pueden superponer en un mismo instante de tiempo.
- Una llamada debe estar preparada para encontrar un estado del objeto que refleje efectos incompletos de otras ejecuciones.

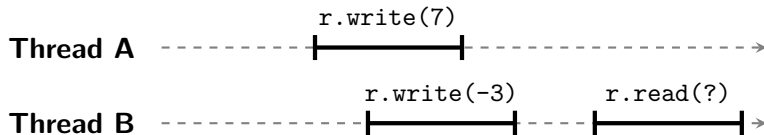
Consistencia Secuencial

- **Las llamadas toman tiempo:** Una **llamada a un método** es el *intervalo* de tiempo que comienza con el evento de **invocación** y concluye con el evento de **retorno**.
- Las llamadas hechas por un proceso son siempre **secuenciales** (no superpuestas).
- Las llamadas hechas por procesos concurrentes pueden **superponerse**.
- Una llamada está **pendiente** si su evento de invocación ocurrió, pero el correspondiente evento de retorno no sucedió.

Consistencia Secuencial: Intuición

Orden secuencial

Las llamadas deben aparentar una ejecución secuencial, una a la vez.

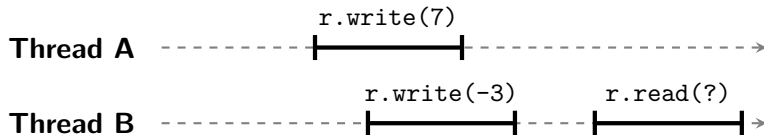


- Se escribe concurrentemente un registro con los valores -3 y 7 .
 - El orden de las operaciones no está definido.
- Luego se lee el registro r . ¿Cuál es el resultado?

Consistencia Secuencial: Intuición

Orden secuencial

Las llamadas deben aparentar una ejecución secuencial, una a la vez.

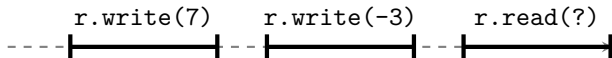


- Se escribe concurrentemente un registro con los valores -3 y 7 .
 - El orden de las operaciones no está definido.
- Luego se lee el registro r . ¿Cuál es el resultado?
 - -3 y 7 son aceptables.
 - -7 no es aceptable (mezcla los efectos de las dos operaciones)

Consistencia Secuencial: Intuición

Orden del Programa (*Program Order*) PO

Es el orden en el que un proceso emite sus llamadas a métodos.



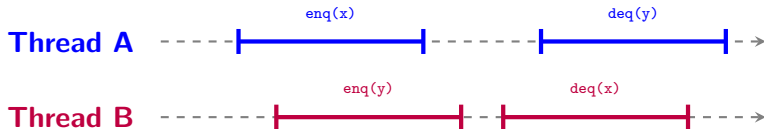
- Un **único proceso** escribe primero 7 y luego -3 y más tarde lee r .
 - Si no hay otros procesos, uno espera leer -3 .
 - La respuesta 7 es inaceptable (en muchos casos)

Consistencia Secuencial

Todas las llamadas a métodos actúan como si hubieran ocurrido en un **orden secuencial consistente con el orden del programa**.

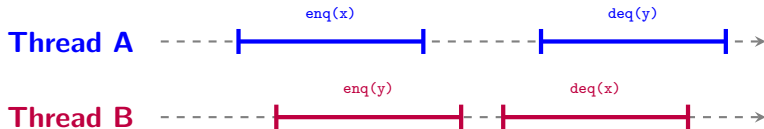
- Debe existir una forma de ordenar todas las llamadas de cualquier ejecución concurrente de modo que cumplan dos condiciones simultáneas:
 - (1) Sean consistentes con el **program order** de cada hilo individual.
 - (2) Cumplan estrictamente con la **especificación secuencial** del objeto compartido.

Ejemplo: Cola FIFO Concurrente



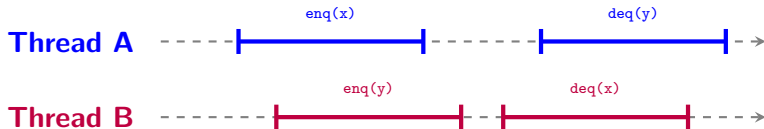
- Dos órdenes secuenciales explican este resultado:
 - Opción 1: $A.enq(x) \rightarrow B.enq(y) \rightarrow B.deq(x) \rightarrow A.deq(y)$
 - Opción 2: $B.enq(y) \rightarrow A.enq(x) \rightarrow A.deq(y) \rightarrow B.deq(x)$

Ejemplo: Cola FIFO Concurrente



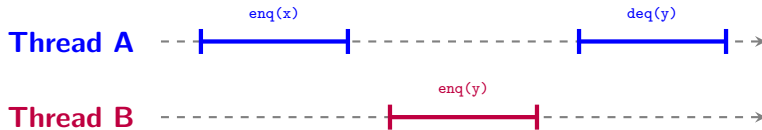
- Dos órdenes secuenciales explican este resultado:

Ejemplo: Cola FIFO Concurrente



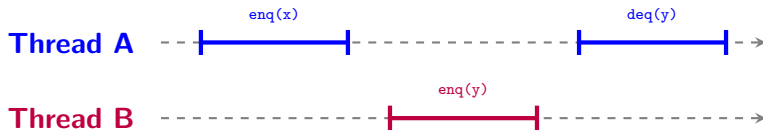
- Dos órdenes secuenciales explican este resultado:
 - Opción 1: $A.enq(x) \rightarrow B.enq(y) \rightarrow B.deq(x) \rightarrow A.deq(y)$
 - Opción 2: $B.enq(y) \rightarrow A.enq(x) \rightarrow A.deq(y) \rightarrow B.deq(x)$

Consistencia Secuencial vs. Tiempo Real



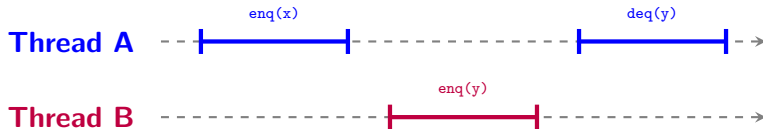
- Satisface consistencia secuencial?

Consistencia Secuencial vs. Tiempo Real



- Satisface consistencia secuencial? Si.
 - Basta ordenar la operación de *B* antes de las de *A*.
 - Notar que encolar *x* ocurrió antes de que comenzara encolar *y*, pero *y* se desencola primero.

Consistencia Secuencial vs. Tiempo Real



- Satisface consistencia secuencial? Si.
 - Basta ordenar la operación de *B* antes de las de *A*.
 - Notar que encolar *x* ocurrió antes de que comenzara encolar *y*, pero *y* se desencola primero.

¿Por qué esto es legal?

`enq(x)` y `enq(y)` ocurren en procesos distintos, **no tienen relación de program order**.

¿Qué es la Composicionalidad?

Una propiedad de correctitud P es **composicional** si, cuando cada objeto del sistema satisface P de manera individual, el sistema también satisface P .

- ¿Por qué es importante?

Permite ensamblar fácilmente un sistema complejo a partir de componentes desarrollados de manera independiente.

- ¿Es la consistencia secuencial composicional?

¿Qué es la Composicionalidad?

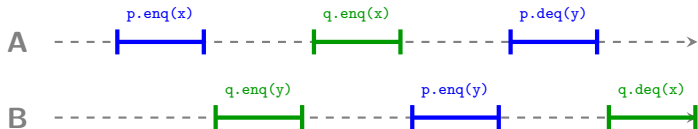
Una propiedad de correctitud P es **composicional** si, cuando cada objeto del sistema satisface P de manera individual, el sistema también satisface P .

- ¿Por qué es importante?

Permite ensamblar fácilmente un sistema complejo a partir de componentes desarrollados de manera independiente.

- ¿Es la consistencia secuencial composicional? **Lamentablemente, NO.**

Consistencia Secuencial NO es Composicional



Cada cola (p y q) es secuencialmente consistente, pero el sistema no

1. Como A desencola y de p : $\langle p.enq(y) B \rangle \rightarrow \langle p.enq(x) A \rangle$
2. Simétricamente, como B desencola x de q : $\langle q.enq(x) A \rangle \rightarrow \langle q.enq(y) B \rangle$
3. Por el **Program Order (PO)** de cada hilo establece que:
 $\langle p.enq(x) A \rangle \rightarrow \langle q.enq(x) A \rangle$ y $\langle q.enq(y) B \rangle \rightarrow \langle p.enq(y) B \rangle$
4. **Contradicción:** Se forma un **ciclo**:
 $\langle p.enq(x) A \rangle \rightarrow \langle q.enq(x) A \rangle \rightarrow \langle q.enq(y) B \rangle \rightarrow \langle p.enq(y) B \rangle \rightarrow \langle p.enq(x) A \rangle$.

Linearizabilidad (Linearizability)

Principio

Cada llamada a un método debe aparentar tener efecto de manera **instantánea** en algún momento entre su invocación y su respuesta.

- **Preservación del Tiempo Real:** Este principio establece que el orden en tiempo real (*real-time order*) **debe ser preservado**.
- **Relación de Inclusión:** Toda ejecución linealizable es secuencialmente consistente, pero **no viceversa**.

Puntos de Linealización

Un **instante específico** donde los efectos de la operación ocurren.

- **Implementaciones basadas en Locks:** Cualquier punto dentro de la sección crítica puede servir como su punto de linealización.
- **Implementaciones sin Locks (*Lock-free*):**
El punto de linealización es típicamente una instrucción (e.g., un CAS) donde los efectos se vuelven visibles para todo el sistema.

Progreso

Es una propiedad de *liveness* que exige a las operaciones **producir respuestas a las invocaciones pendientes**.

- Idealmente queremos que *toda* invocación pendiente obtenga una respuesta.
- Esto no es posible si los procesos que contienen las invocaciones pendientes **dejan de ejecutar** (por ejemplo, se mueren).
- **Suposición:** Se exige progreso para los procesos que se mantienen ejecutando de manera activa.

Condiciones de Progreso Bloqueantes

Dependencia de fairness

Asumimos que el scheduler es fair.

Libre de Inanición (*Starvation-free*)

Un método está *libre de inanición* si toda llamada finaliza en una cantidad finita de pasos (si **los procesos** con llamadas pendientes continúen).

- Garantiza un progreso individual, condicionada a la ejecución de los demás.

Libre de *Deadlock*

Un método es libre de *deadlock* si **al menos alguna llamada finaliza** en una cantidad finita de pasos (si los procesos con llamadas pendientes ejecutan).

- Garantiza únicamente un progreso mínimo global para todo el sistema.

Condiciones de Progreso No Bloqueantes: Wait-freedom

Condición No Bloqueante

Un retraso arbitrario por parte de un proceso no puede impedir que los demás procesos sigan progresando de manera independiente.

Método Wait-free

Un método es *wait-free* si cualquier llamada pendiente puede completar en una cantidad finita de pasos, **independientemente de las ejecuciones de los demás**.

Implementación de un objeto es Wait-free

La implementación de Un objeto es *wait-free* si todos sus métodos son wait-free.

Condiciones de Progreso: Lock-freedom

Lock-freedom

Un método es *lock-free* si su ejecución garantiza que al menos una llamada pendiente finalice en una cantidad finita de pasos.

- **Riesgo de Inanición (*Starvation*):**
Admite la posibilidad de que algunos procesos sufran inanición.
- Toda implementación *wait-free* es *lock-free* (no viceversa)

Interfaz Set

```
public interface Set {  
    public boolean add(Object o);  
    public boolean remove(Object o);  
    public boolean contains(Object o);  
}
```

Implementación como lista simplemente enlazada

Representación

```
public class Node {  
    public Object item;  
    public int key;  
    public Node next;  
    public Node (int key){...}  
}  
  
public class List {  
    Node head;  
}
```

Implementación como lista simplemente enlazada

Representación

```
public class Node {  
    public Object item;  
    public int key;  
    public Node next;  
    public Node (int key){...}  
}  
  
public class List {  
    Node head;  
}
```

Invariante

- key es el hash-code del elemento item
- No existen claves duplicadas
- Nodos ordenados ascendentemente por key
- Dos **nodos centinelas**.
 - Marcan el inicio y el fin de la lista.
 - No contienen elementos.

Implementación como lista simplemente enlazada

Representación

```
public class Node {  
    public Object item;  
    public int key;  
    public Node next;  
    public Node (int key){...}  
}  
  
public class List {  
    Node head;  
}
```

Invariante

- `key` es el hash-code del elemento `item`
- No existen claves duplicadas
- Nodos ordenados ascendentemente por `key`
- Dos **nodos centinelas**.
 - Marcan el inicio y el fin de la lista.
 - No contienen elementos.

Función de abstracción

Un elemento pertenece al conjunto si y solo si está en un nodo (distinto de `head` y `tail`) alcanzable desde `head`

Implementación como lista simplemente enlazada

Constructor

```
public class List {  
    Node head;  
    public List(){  
        head = new Node (Integer.MIN_VALUE);  
        head.next = new Node (Integer.MAX_VALUE);  
    }  
}
```

Sincronización de granularidad gruesa

- Un lock para toda la estructura (e.g., un monitor).
- Simple y correcto.
- No funciona bien con contención.

Método add

```
public synchronized add (Object o) {  
    Node pred, curr;  
    int key = o.hashCode();  
    pred = head;  
    curr = pred.next;  
    while (curr.key < key){  
        pred = curr;  
        curr = curr.next;  
    }  
    if (key == curr.key) return false;  
    else {  
        Node node = new Node (o);  
        node.next = curr;  
        pred.next = node;  
        return true;  
    }  
}
```

Sincronización de granularidad gruesa

- **Un lock para toda la estructura** (e.g., un monitor).
- Simple y correcto.
- No funciona bien con contención.

Método remove

```
public synchronized remove (Object o) {  
    Node pred, curr;  
    int key = o.hashCode();  
    pred = head;  
    curr = pred.next;  
    while (curr.key <= key){  
        if (o == curr.item) {  
            pred.next = curr.next;  
            return true;  
        }  
        pred = curr;  
        curr = curr.next;  
    }  
    return false;  
}
```

Granularidad gruesa: Análisis

Correctitud : Linearizabilidad

Garantizar que la estructura es **linealizable**.

- **Punto de Linearizabilidad:** Un paso atómico (lectura, escritura o CAS) donde la operación “tiene efecto”.
- **Función de Abstracción:** Al aplicar la función de abstracción en un punto de linealización, el resultado debe definir una **ejecución secuencial válida** de un conjunto (add, remove, contains).

Granularidad gruesa: Análisis

Correctitud : Linearizabilidad

Garantizar que la estructura es **linearizable**.

- **Punto de Linearizabilidad:** Un paso atómico (lectura, escritura o CAS) donde la operación “tiene efecto”.
- **Función de Abstracción:** Al aplicar la función de abstracción en un punto de linealización, el resultado debe definir una **ejecución secuencial válida** de un conjunto (add, remove, contains).

Puntos de Linearización

- **add(a) o remove(a) Exitosos:** Se linearizan cuando actualizan el atributo next del nodo predecesor.
- **add(a) / remove(a) Inexitosos y contains(a):** Se pueden linearizar en el momento en que adquieren el lock (o en cualquier instante mientras se retenga).

Garantías de Progreso

Diferentes algoritmos de listas imponen diferentes compromisos:

- **Algoritmos Bloqueantes (Con Locks):**
Requieren analizar la ausencia de deadlocks (*deadlock-free*) e inanición (*starvation-free*).
- **Algoritmos No Bloqueantes (*Nonblocking*):**
 - ***Wait-free*:** Cada llamada pendiente se completa de forma individual en una cantidad finita de pasos.
 - ***Lock-free*:** Alguna llamada pendiente siempre finaliza, garantizando el progreso global del sistema.

Granularidad gruesa: Progreso

Garantías de Progreso

Diferentes algoritmos de listas imponen diferentes compromisos:

- **Algoritmos Bloqueantes (Con Locks):**
Requieren analizar la ausencia de deadlocks (*deadlock-free*) e inanición (*starvation-free*).
- **Algoritmos No Bloqueantes (*Nonblocking*):**
 - ***Wait-free*:** Cada llamada pendiente se completa de forma individual en una cantidad finita de pasos.
 - ***Lock-free*:** Alguna llamada pendiente siempre finaliza, garantizando el progreso global del sistema.

Garantía de Progreso

Es libre de inanición si su lock lo es.

Sincronización de granularidad fina

- **Dividir una estructura en partes**, donde cada parte tiene su propio lock.
- Operaciones que actúan sobre distintas partes no necesitan ejecutar en exclusión mutua.
- Los algoritmos se tienen que pensar cuidadosamente.

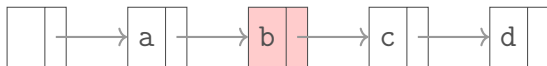
Sincronización de granularidad fina

Clase Node

```
public class Node {  
    public Object item;  
    public int key;  
    public Node next;  
    public lock();  
    public unlock ();  
}
```

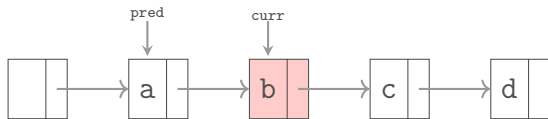
- Se agrega un lock en cada nodo con los métodos correspondientes.
- En Java se heredan de Object
 - Cada objeto tiene un lock reentrante incorporado (lock de monitor)

Granularidad fina: Eliminación de un nodo



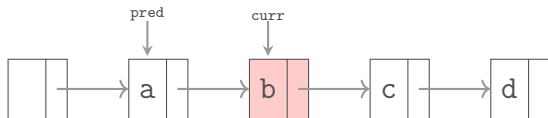
- Queremos eliminar al nodo **b**.

Granularidad fina: Eliminación de un nodo



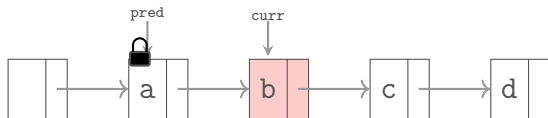
- Se recorre la lista hasta llegar a **b**.

Granularidad fina: Eliminación de un nodo



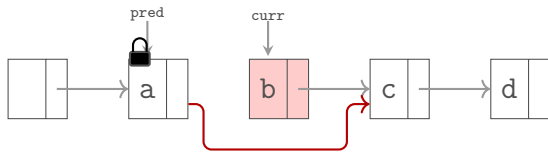
- Se debe modificar el puntero `next` del predecesor **a** para hacerlo apuntar a **c**.

Granularidad fina: Eliminación de un nodo



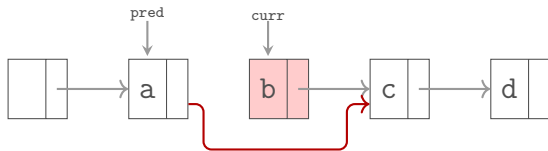
- **Regla:** Se adquiere el **lock** del nodo apuntado por $pred$ antes de modificar.

Granularidad fina: Eliminación de un nodo



- Se modifica la lista.

Granularidad fina: Eliminación de un nodo



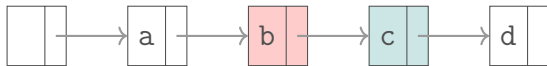
- Se libera el lock.

Granularidad fina: Eliminación de un nodo



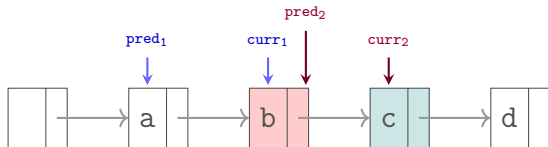
- Se libera la memoria del nodo eliminado.

Dos eliminaciones concurrentes



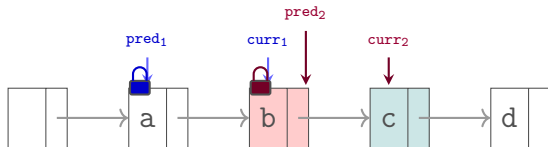
- El **Thread 1** quiere borrar **b** (bloquea **a** con pred_1).
- El **Thread 2** quiere borrar **c** (bloquea **b** con pred_2).

Dos eliminaciones concurrentes



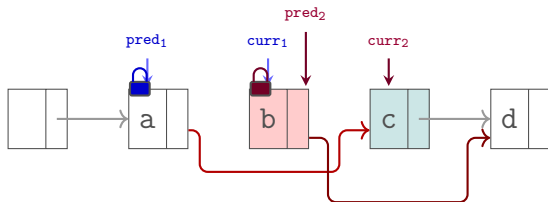
- Los dos procesos recorren la lista hasta llegar al nodo a eliminar.

Dos eliminaciones concurrentes



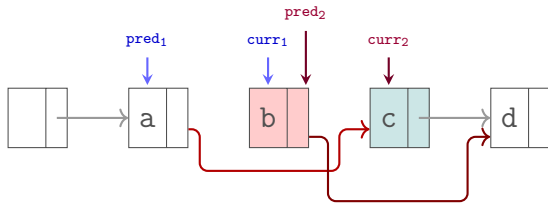
- Los dos procesos toma el lock sobre los nodos a modificar.

Dos eliminaciones concurrentes



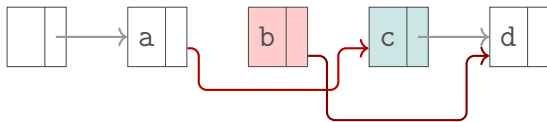
- Se modifican los punteros para eliminar los nodos.

Dos eliminaciones concurrentes



- **Problema:** Al borrar **b**, el puntero de **a** pasa a apuntar a **c**.
- ¡Pero el nodo **c** fue removido por el Thread 2! Esta eliminación se pierde.

Dos eliminaciones concurrentes



- **Problema:** Al borrar **b**, el puntero de **a** pasa a apuntar a **c**.
- ¡Pero el nodo **c** fue removido por el Thread 2! Esta eliminación se pierde.

Dos eliminaciones concurrentes



- **Problema:** Al borrar **b**, el puntero de **a** pasa a apuntar a **c**.
- ¡Pero el nodo **c** fue removido por el Thread 2! Esta eliminación se pierde.

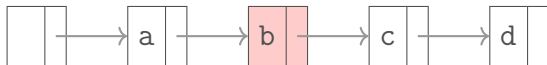
Solución: Intuición

- Cuando se tiene el lock sobre un nodo para modificar, no puede eliminarse el nodo sucesor.

Solución: Intuición

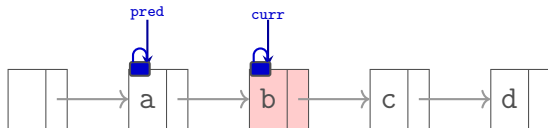
- Cuando se tiene el lock sobre un nodo para modificar, no puede eliminarse el nodo sucesor.
- Adquirir también el lock sobre el nodo que va a ser eliminado.

Eliminar un nodo (Lock Coupling)



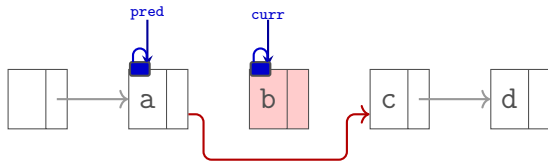
- Al eliminar un nodo se deben adquirir

Eliminar un nodo (Lock Coupling)



- Al eliminar un nodo se deben adquirir
 - el lock sobre el nodo predecesor
 - el lock sobre el nodo a eliminar (garantiza que el sucesor no puede ser eliminado concurrentemente)

Eliminar un nodo (Lock Coupling)



- Al eliminar un nodo se deben adquirir
 - el lock sobre el nodo predecesor
 - el lock sobre el nodo a eliminar (esto garantiza que el sucesor no puede ser eliminado concurrentemente)
- Un thread debe adquirir el lock sobre un nodo cuando tiene el lock sobre el nodo predecesor (hand-over-hand locking / lock coupling)

Remove - Versión secuencial

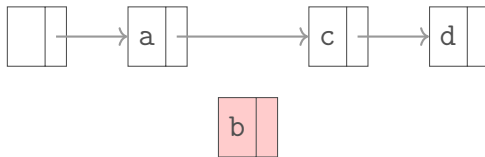
```
public boolean remove(Object o) {  
    Node pred, curr;  
    int key = o.hashCode();  
    pred = head;  
    curr = pred.next;  
    while (curr.key <= key) {  
        if (o == curr.item) {  
            pred.next = curr.next;  
            return true;  
        }  
        pred = curr;  
        curr = curr.next;  
    }  
    return false;  
}
```

Lock hand-over-hand (Remove)

```
public boolean remove(T item) {  
    int key = item.hashCode();  
    head.lock();  
    Node pred = head;  
    try {  
        Node curr = pred.next;  
        curr.lock();  
        try {  
            while (curr.key < key) {  
                pred.unlock();  
                pred = curr;  
                curr = curr.next;  
                curr.lock();  
            }  
            // Continúa a la derecha...
```

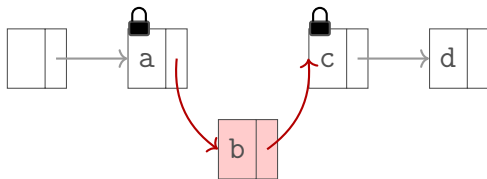
```
// ... Viene de la izquierda  
        if (curr.key == key) {  
            pred.next = curr.next;  
            return true;  
        }  
        return false;  
    } finally {  
        curr.unlock();  
    }  
} finally {  
    pred.unlock();  
}  
}
```

Agregar un nodo



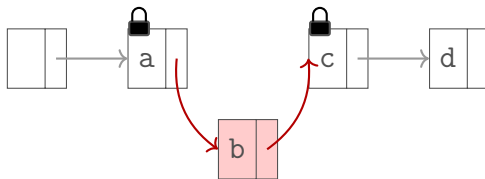
- Al agregar un nodo se debe adquirir el

Agregar un nodo



- Al agregar un nodo se debe adquirir el
 - lock sobre el nodo predecesor

Agregar un nodo



- Al agregar un nodo se debe adquirir el
 - lock sobre el nodo predecesor
 - lock sobre el nodo sucesor

add - Versión secuencial

```
public boolean add(Object o) {  
    Node pred, curr;  
    int key = o.hashCode();  
    pred = head;  
    curr = pred.next;  
    while (curr.key < key) {  
        pred = curr;  
        curr = curr.next;  
    }  
    if (key == curr.key) return false;  
    else {  
        Node node = new Node(o);  
        node.next = curr;  
        pred.next = node;  
        return true;  
    }  
}
```

Lock hand-over-hand (add)

```
public boolean add(T item) {
    int key = item.hashCode();
    head.lock();
    Node pred = head;
    try {
        Node curr = pred.next;
        curr.lock();
        try {
            while (curr.key < key) {
                pred.unlock();
                pred = curr;
                curr = curr.next;
                curr.lock();
            }
            // Continúa a la derecha
            ...
        }
    }
```

```
// ... Viene de la izquierda
if (curr.key == key) {
    return false;
}
Node node = new Node(item);
node.next = curr;
pred.next = node;
return true;
} finally {
    curr.unlock();
}
} finally {
    pred.unlock();
}
}
```

Granularidad Fina: Progreso (ausencia de deadlocks)

Ausencia de deadlocks

Para evitar deadlocks, **todos los métodos adquieren los locks exactamente en el mismo orden.**

- La adquisición comienza en el nodo head.
- Luego se adquieren siguiendo las referencias next (un orden total).

Granularidad Fina: Progreso (libre de inanición)

Inanición

Es libre de inanición si los locks son fair.

Demostración

- **Éxito garantizado en la cabeza:** Si un thread intenta adquirir head, eventualmente lo conseguirá
- Como no existen deadlocks, todos los locks tomados hacia adelante por otros procesos serán eventualmente liberados.
- Por lo tanto, el thread avanzará y logrará bloquear los nodos pred y curr.

Granularidad Fina: Correctitud

Puntos de linearización

- `add(a)` y `remove(a)` **Exitosos**: cuando actualiza el campo `next` del nodo predecesor para enlazar al nuevo nodo.
- Cualquier llamada a `contains(a)`, o un `add(a)` / `remove(a)` inexitoso, cuando adquiere el lock de un nodo cuya clave es mayor o igual a a .

Características

- Mejora la solución de granularidad gruesa
 - Muchos threads pueden manipular la estructura concurrentemente
- Ineficiente: larga cadena de adquisición y liberación de locks

Estrategia

1. **Buscar:** Recorrer la lista sin adquirir locks.
2. **Bloquear:** Una vez localizados los nodos (pred y curr), adquirir sus locks.

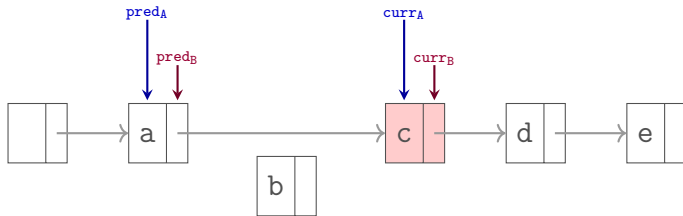
Remove - Tentativa

```
public boolean remove(Object o) {
    Node pred, curr;
    int key = o.hashCode();
    pred = head;
    curr = pred.next;
    while (curr.key < key) {
        pred = curr;
        curr = curr.next;
    }
    try {
        pred.lock(); curr.lock();
        if (curr.item == o) {
            pred.next = curr.next;
            return true;
        } else
            return false;
    } finally { pred.unlock(); curr.unlock(); }
}
```

Add - Tentativa

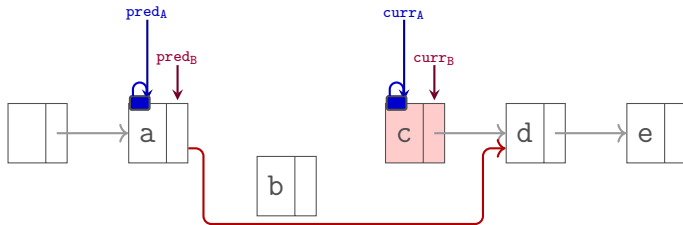
```
public boolean add(Object o) {
    Node pred, curr;
    int key = o.hashCode();
    pred = head;
    curr = pred.next;
    while (curr.key < key) {
        pred = curr;
        curr = curr.next;
    }
    try {
        pred.lock(); curr.lock();
        if (key == curr.key) return false;
        else {
            Node node = new Node(o);
            node.next = curr;
            pred.next = node;
            return true;
        }
    } finally { pred.unlock(); curr.unlock(); }
}
```

Eliminar c y agregar b



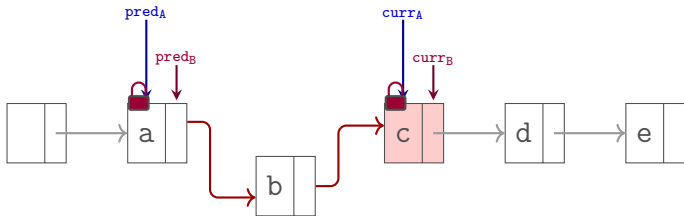
- Ambos hilos recorren la lista de forma optimista sin adquirir ningún lock.
- Identifican las posiciones locales para sus respectivas operaciones.

Eliminar c y agregar b



- El **Thread 1 (A)** adquiere los locks sobre su predecesor y objetivo.
- Borra exitosamente el nodo **c** modificando el puntero para apuntar a **d**.

Eliminar c y agregar b



- El **Thread 2 (B)** adquiere locks sobre **a** y **c** (sin saber que ya fueron modificados).
- Enlaza el nuevo nodo **b**. Resultado: se agregó b pero no se eliminó c.

Sincronización Optimista

Estrategia

Tres pasos ejecutados suponiendo que no habrá colisiones:

1. **Buscar:** Recorrer la lista sin adquirir locks.
2. **Bloquear:** Una vez localizados los nodos (`pred` y `curr`), adquirir sus locks.
3. **Validar:** Verificar que los nodos bloqueados son válidos.

Manejo de Conflictos

Si hubo cambios (los nodos bloqueados no son válidos), volver a empezar liberando los locks y reiniciar.

Validación

```
private boolean validate(Node pred, Node curr){  
    Node aux = head;  
    while (aux.key <= pred.key) {  
        if (aux == pred) //accessible  
            return pred.next == curr; //adyacentes  
        aux = aux.next;  
    }  
    return false;  
}
```

Lock optimista (remove)

```
public boolean remove(T item) {  
    int key = item.hashCode();  
    while (true) {  
        Node pred = head;  
        Node curr = pred.next;  
        // Recorrido sin adquirir  
locks  
        while (curr.key < key) {  
            pred = curr;  
            curr = curr.next;  
        }  
        pred.lock();  
        try {  
            curr.lock();  
            try {  
                // Continúa a la derecha...
```

```
        // ... Viene de la izquierda  
        if (validate(pred, curr)) {  
            if (curr.key == key) {  
                pred.next = curr.next;  
                return true;  
            } else {  
                return false;  
            }  
        }  
    } finally {  
        curr.unlock();  
    }  
} finally {  
    pred.unlock();  
}  
}  
}
```

Correctitud (remove)

- Si
 - se tienen los locks de los dos nodos **a** y **c**
 - si **a** todavía es accesible desde head
 - si **c** es el sucesor de **a**
- entonces
 - **a** y **c** no pueden ser eliminados
 - Se puede borrar y retornar true

Lock optimista (add)

```
public boolean add(T item) {
    int key = item.hashCode();
    while (true) {
        Node pred = head;
        Node curr = pred.next;
        // Recorrido sin adquirir
        locks
        while (curr.key < key) {
            pred = curr;
            curr = curr.next;
        }
        pred.lock();
        try {
            curr.lock();
            try {
                // Continúa a la derecha
                ...
            }
        }
    }
}
```

```
// ... Viene de la izquierda
if (validate(pred, curr)) {
    if (curr.key == key) {
        return false;
    } else {
        Node node = new Node(item);
        node.next = curr;
        pred.next = node;
        return true;
    }
}
} finally {
    curr.unlock();
}
} finally {
    pred.unlock();
}
}
}
```

Lock optimista (contains)

```
public boolean contains(T item) {  
    int key = item.hashCode();  
    while (true) {  
        Node pred = head;  
        Node curr = pred.next;  
        // Búsqueda sin locks  
        while (curr.key < key) {  
            pred = curr;  
            curr = curr.next;  
        }  
        pred.lock();  
        try {  
            curr.lock();  
            try {  
                // Continúa a la derecha  
                ...  
            }  
        }  
    }  
}
```

```
        // ... Viene de la izquierda  
        if ( validate(pred, curr) ) {  
            return (curr.key == key);  
        }  
    } finally {  
        curr.unlock();  
    }  
} finally {  
    pred.unlock();  
}  
}
```

Características

- Menos adquisiciones y liberaciones de locks
 - Mayor performance
 - Mayor concurrencia
- Requiere recorrer la lista dos veces (una vez adicional para la verificación)
- El método contains necesita de la adquisición de locks

Sincronización Optimista: Correctitud

Recorrido sin Locks e Interferencia

Al recorrer sin tomar locks, un proceso puede atravesar nodos eliminados.

- **Ausencia de interferencia:** cuando un nodo es desvinculado, `next` no cambia.
- Garbage collector garantiza que un nodo no es reciclado mientras está siendo usado
- Una secuencia de punteros huérfanos eventualmente redirige a la lista.

Puntos de Linealización (condicionales)

Idénticos a los de la granularidad fina, pero considerando validaciones fallidas:

- Una llamada a un método se linealiza cuando adquiere locks y la *validación* posterior es exitosa.

Sincronización Optimista: Progreso

Libre de deadlocks

Porque los deadlocks se toman en orden creciente.

Posibilidad de Inanición

NO es libre de inanición incluso si todos los locks de los nodos son fair.

- Un proceso puede ser relegado indefinidamente si otros procesos agregan y eliminan nodos repetidamente de forma concurrente.

Evaluación en la práctica

- El fenómeno de la inanición es infrecuente.
- Funciona mejor si el costo de recorrer dos veces sin usar locks es significativamente menor que el costo de recorrer una sola vez tomando locks.

Sincronización Lazy

- Antes de borrar marcar cada nodo como borrado
- No se necesita recorrer la lista dos veces (para validar)
- El remove es lazy:
 - Primero marca el nodo a borrar (lo remueve lógicamente)
 - Luego cambia el puntero next
- Extendemos la representación de los nodos con un campo **marked** (inicialmente false)

Sincronización Lazy: Implementación

Representación

```
public class Node {
    public Object item;
    public int key;
    public Node next;
    public Node (int key){...}
    public lock();
    public unlock ();
    public boolean marked = false;
}

public class List {
    Node head;
}
```

Invariante

- key es el hash-code del elemento item
- No existen claves duplicadas
- Nodos ordenados ascendentemente por key
- Dos **nodos centinelas**.
 - Marcan el inicio y el fin de la lista.
 - No contienen elementos.

Función de abstracción

Un elemento pertenece al conjunto si y solo si está en un nodo (distinto de head y tail) alcanzable desde head y no está marcado.

Validación

```
private boolean validate(Node pred, Node curr) {  
    return !pred.marked && !curr.marked && pred.next == curr;  
}
```

Sincronización Lazy (remove)

```
public boolean remove(T item) {
    int key = item.hashCode();
    while (true) {
        Node pred = head;
        Node curr = head.next;
        while (curr.key < key) {
            pred = curr;
            curr = curr.next;
        }
        pred.lock();
        try {
            curr.lock();
            try {
                // Continúa a la derecha
                ...
            }
        }
    }
}
```

```
// ... Viene de la izquierda
if (validate(pred, curr)) {
    if (curr.key == key) {
        // Remoción lógica
        curr.marked = true;
        // Remoción física
        pred.next = curr.next;
        return true;
    } else {
        return false;
    }
} finally {
    curr.unlock();
} finally {
    pred.unlock();
}
}
```

Sincronización Lazy (add)

```
public boolean remove(T item) {  
    int key = item.hashCode();  
    while (true) {  
        Node pred = head;  
        Node curr = head.next;  
        while (curr.key < key) {  
            pred = curr;  
            curr = curr.next;  
        }  
        pred.lock();  
        try {  
            curr.lock();  
            try {  
                // Continúa a la derecha  
                ...  
            }  
        }  
    }  
}
```

```
// ... Viene de la izquierda  
if ( validate(pred, curr) ) {  
    if (curr.key == key) {  
        return false;  
    } else {  
        Node node = new Node (o);  
        node.next = curr;  
        pred.next = node;  
        return true;  
    }  
}  
} finally {  
    curr.unlock();  
}  
} finally {  
    pred.unlock();  
}  
}  
}
```

Sincronización Lazy (contains)

```
public boolean contains(T item) {  
    int key = item.hashCode();  
    Node curr = head;  
    while (curr.key < key)  
        curr = curr.next;  
    return curr.key == key && !curr.marked;  
}
```

Garantías de Progreso en `add()` y `remove()`

Al igual que en el algoritmo optimista, los métodos `add()` y `remove()` **NO son libres de inanición**.

- Los recorridos de un proceso pueden demorarse indefinidamente por modificaciones concurrentes continuas.

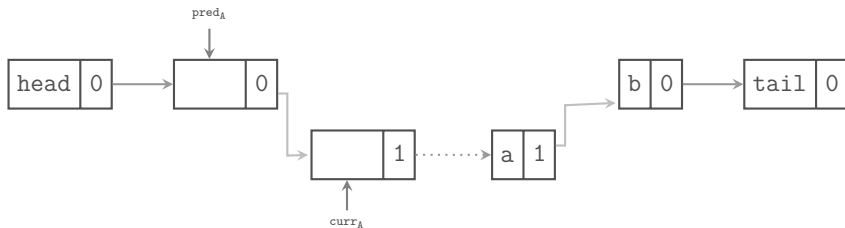
`contains()` es Wait-free

El método recorre la lista una sola vez, ignorando por completo los locks. Retorna `true` si el nodo buscado está presente y no marcado; de lo contrario, retorna `false`.

Puntos de Linearización de `add()`, `remove()` y `contains()` Exitoso

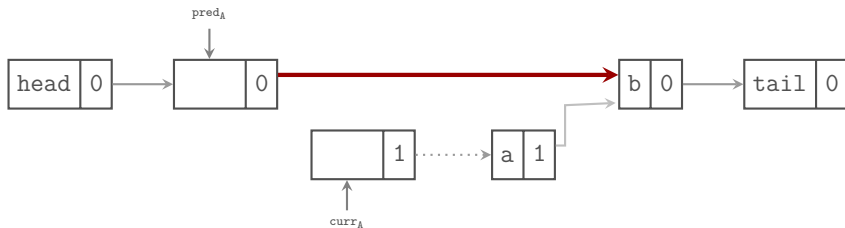
- `add()` y `remove()` **Inexitoso**: Mismo criterio que en el algoritmo optimista (en el instante en que se adquieren los locks y la validación resulta exitosa).
- `remove()` **Exitoso**: Se lineariza exactamente cuando el nodo es marcado lógicamente (en el instante en que se setea el bit `marked`).
- `contains()` **Exitoso**: Se lineariza en el momento en que se localiza el nodo coincidente no marcado.

Linealización del contains() no existoso



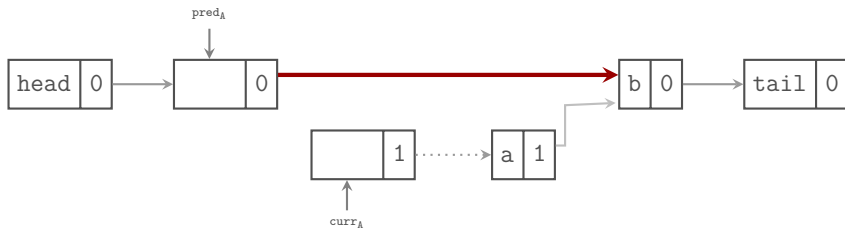
- El proceso A busca a , recorre una sección eliminada (lógicamente) de la lista.

Linealización del contains() no existoso



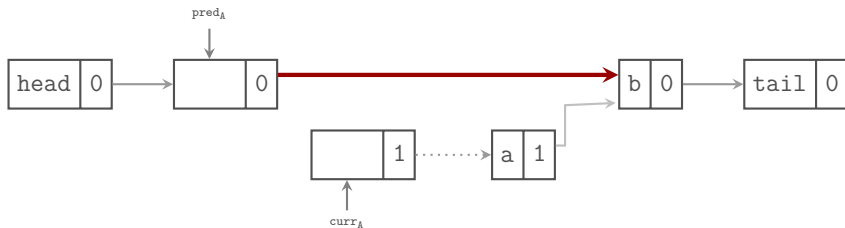
- Otro proceso elimina físicamente la parte eliminada.

Linealización del contains() no existoso



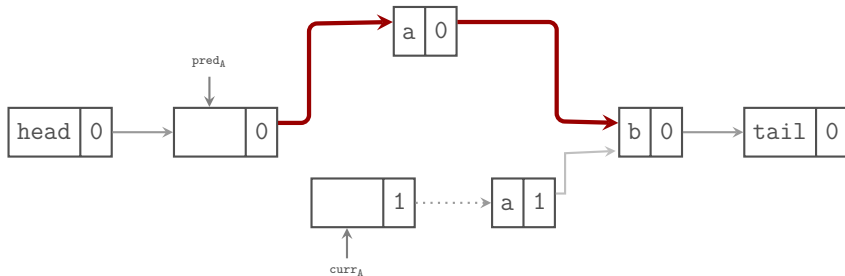
- Cuando *A* retoma, detecta que *a* no pertenece al conjunto.

Linealización del contains() no existoso



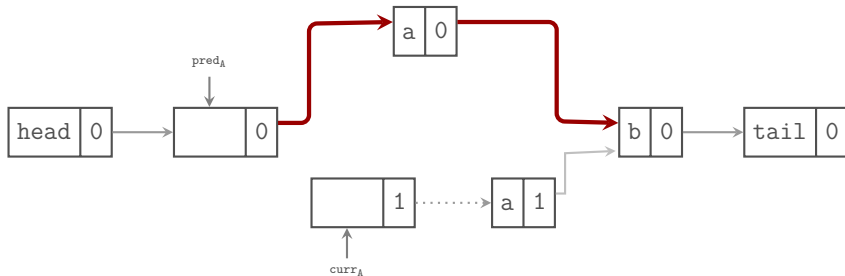
- Podemos linealizar la llamada en el punto en el que *A* detecta que *a* está marcado y no pertenece al conjunto.

Linealización del contains() no existoso



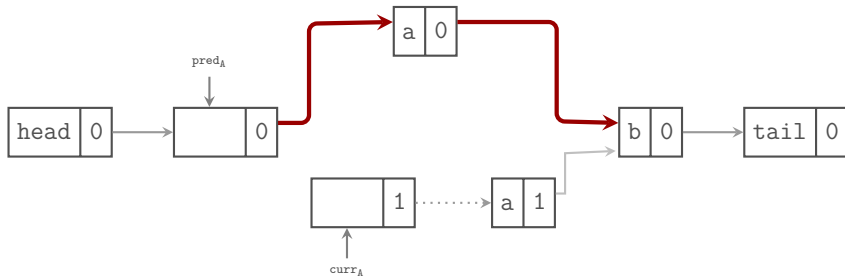
- Otro proceso podría llamar a `add(a)`, insertando un nuevo nodo válido con clave `a` en la parte alcanzable.

Linealización del contains() no existoso



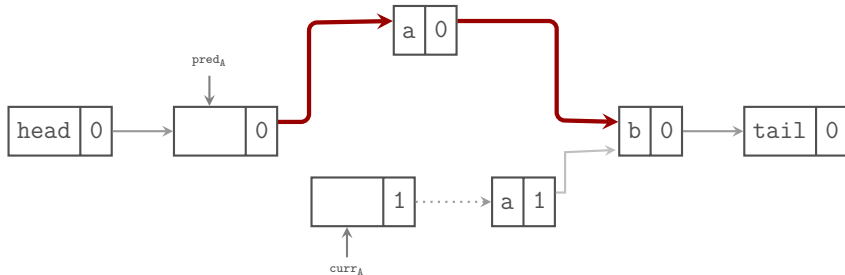
- No podemos linealizar a A cuando encuentra la marca, porque ocurre *después* de la nueva inserción.

Linealización del contains() no existoso



- El punto es **inmediatamente anterior** a que el nuevo nodo con clave `a` sea añadido a la lista por el otro hilo concurrentemente.

Linealización del contains() no existoso



- El punto es **inmediatamente anterior** a que el nuevo nodo con clave *a* sea añadido a la lista por el otro hilo concurrentemente.
- El punto de linealización está determinado por el orden global de eventos; ocurre de forma dinámica y puede no coincidir con un paso propio del proceso.

Sincronización Optimista: Características

- Contains no necesita locks
- Se evita el doble recorrido de la lista para las operaciones no conflictivas
- Las operaciones con contención necesitan recorrer dos veces la lista
- Uso de locks es crítico si un thread no lo devuelve

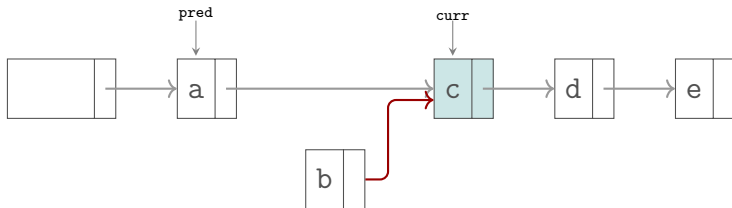
Sincronización No Bloqueante

- Las operaciones con contención necesitan recorrer dos veces la lista
- Eliminar completamente locks
- Usar sólo **CompareAndSet()**

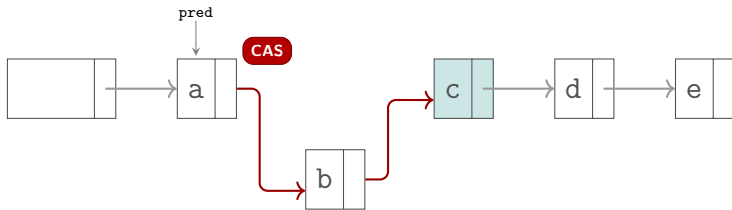
Sincronización No Bloqueante: Agregar un nodo (naïve)



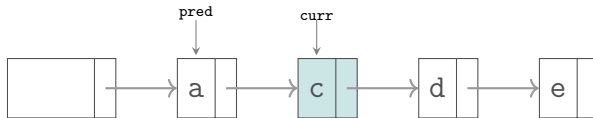
Sincronización No Bloqueante: Agregar un nodo (naïve)



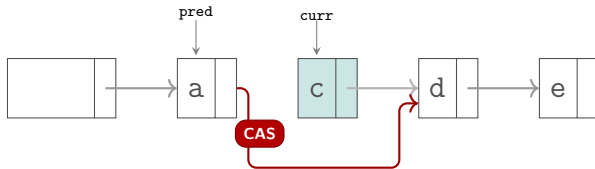
Sincronización No Bloqueante: Agregar un nodo (naïve)



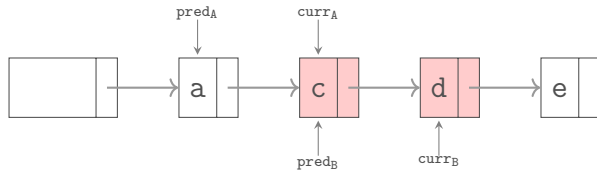
Sincronización No Bloqueante: Eliminar un nodo (naïve)



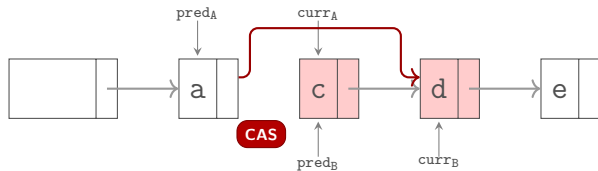
Sincronización No Bloqueante: Eliminar un nodo (naïve)



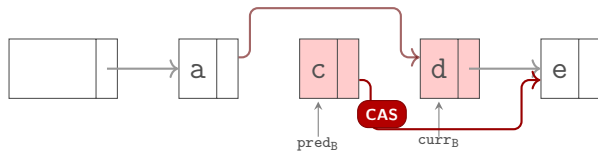
Sincronización No Bloqueante: Eliminación Concurrente



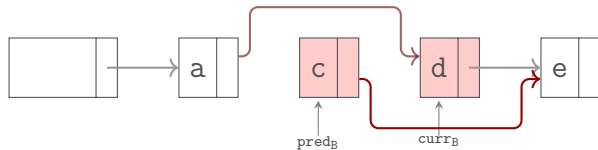
Sincronización No Bloqueante: Eliminación Concurrente



Sincronización No Bloqueante: Eliminación Concurrente



Sincronización No Bloqueante: Eliminación Concurrente



Déjà-vu!!!

Sincronización No Bloqueante : solución

- Se actualiza un nodo (el campo next) luego de que fue eliminado (nodo **c**)
- Usar eliminación lazy
 - Primero marcar que el nodo fue removido
 - Después cambiar el puntero
- CAS (modificado)
 - los campos next y marked del nodo se tratan como una única unidad atómica: cualquier intento de actualizar el campo next cuando el campo marked es verdadero (true) falla.

Interfaz Referencias marcadas

```
interface ReferenciaMarcada<T>{  
  
    public boolean compareAndSet(T expectedReference,  
                                T newReference,  
                                boolean expectedMark,  
                                boolean newMark);  
  
    public T get(boolean[] marked); // marked is output  
    public T getReference();  
    public boolean isMarked();  
}
```

Comportamiento de compareAndSet

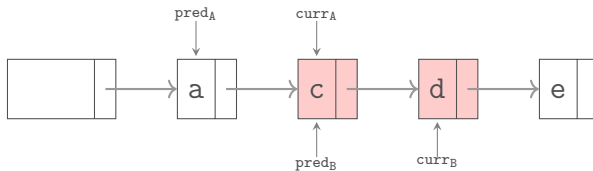
Verifica de forma atómica que la referencia y la marca actuales coincidan con los valores esperados (expectedReference y expectedMark). Si ambas condiciones se cumplen, los reemplaza por los nuevos valores actualizados.

Sincronización No Bloqueante: Implementación

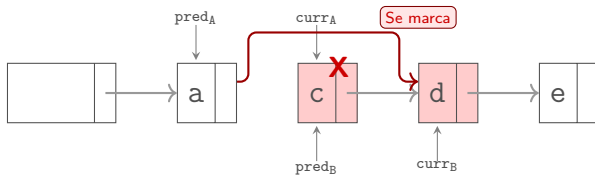
Representación

```
public class Node {  
    public Object item;  
    public int key;  
    public ReferenciaMarcada<Node> next;  
    public Node (int key){...}  
}  
  
public class List {  
    ReferenciaMarcada<Node> head;  
}
```

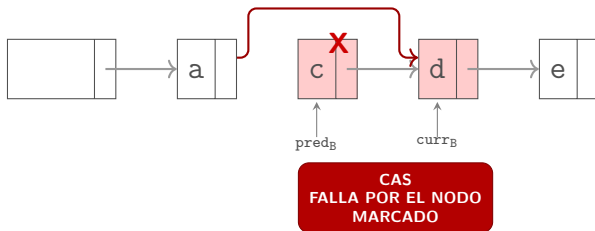
Eliminación Concurrente



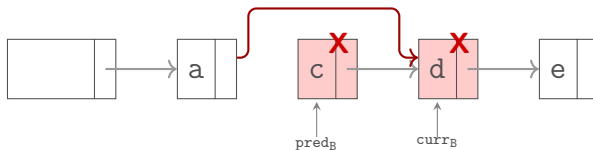
Eliminación Concurrente



Eliminación Concurrente



Eliminación Concurrente



- Nodo c se elimina físicamente.
- Nodo d se elimina lógicamente (marcado)
 - Será removido por las subsiguientes operaciones

Algoritmo Lock-Free: Limpieza cooperativac

Limitación

No es seguro cambiar `next` de un nodo ya marcado (ejemplo, eliminación concurrente). Si un hilo detecta esto, debe reiniciar el recorrido desde `head`.

Algoritmo Lock-Free: Limpieza cooperativac

Limitación

No es seguro cambiar `next` de un nodo ya marcado (ejemplo, eliminación concurrente). Si un hilo detecta esto, debe reiniciar el recorrido desde `head`.

Estrategia de Recorrido Cooperativo

Para evitar reintentos:

- `add()` y `remove()`: Limpian físicamente cualquier nodo marcado que encuentren en su camino hacia el objetivo.
- `contains()`: Al ser de solo lectura, no participa en la limpieza y puede atravesar nodos marcados y no marcados sin detenerse.

Algoritmo Lock-Free: Clase Window

Abstracción

- Se factoriza la lógica de búsqueda y de pred y curr y limpieza.
- El método `find()` avanza por la lista y, cada vez que encuentra un nodo marcado, intenta un `compareAndSet()` para removerlo físicamente.
- Si falla debido a cambios concurrentes, reinicia el recorrido desde `head`.

Estructura Window y Método find

```
class Window {
    public Node pred, curr;
    Window(Node myPred, Node myCurr) {
        pred = myPred; curr = myCurr;
    }
}

public Window find(Node head, int key) {
    Node pred = null, curr = null,
        succ = null;
    boolean[] marked = {false};
    boolean snip;
    retry: while (true) {
        pred = head;
        curr = pred.next.getReference();
        while (true) {
            // Continúa a la derecha...
```

```
// ... Viene de la izquierda
        succ = curr.next.get(marked);
        while (marked[0]) {
            snip = pred.next.compareAndSet(
                curr, succ, false, false);
            if (!snip) continue retry;
            curr = succ;
            succ = curr.next.get(marked);
        }
        if (curr.key >= key)
            return new Window(pred, curr);
        pred = curr;
        curr = succ;
    }
}
```

Algoritmo Lock-Free: Método add

```
public boolean add(T item) {  
    int key = item.hashCode();  
    while (true) {  
        Window window = find(head, key);  
        Node pred = window.pred, curr = window.curr;  
        if (curr.key == key) {  
            return false;  
        } else {  
            Node node = new Node(item);  
            node.next = new AtomicMarkableReference(curr, false);  
            if (pred.next.compareAndSet(curr, node, false, false)) {  
                return true;  
            }  
        }  
    }  
}
```

Algoritmo Lock-Free: Método remove

```
public boolean remove(T item) {  
    int key = item.hashCode();  
    boolean snip;  
    while (true) {  
        Window window = find(head, key);  
        Node pred = window.pred, curr = window.curr;  
        if (curr.key != key) {  
            return false;  
        } else {  
            Node succ = curr.next.getReference();  
            snip = curr.next.compareAndSet(succ, succ, false, true);  
            if (!snip)  
                continue;  
            pred.next.compareAndSet(curr, succ, false, false);  
            return true;  
        }  
    }  
}
```

Algoritmo Lock-Free: Método contains

```
public boolean contains(T item) {  
    int key = item.hashCode();  
    Node curr = head;  
    while (curr.key < key) {  
        curr = curr.next.getReference();  
    }  
    return (curr.key == key && !curr.next.isMarked());  
}
```

Algoritmo Lock-Free: Puntos de Linealización

Operaciones Exitosas (Modificaciones)

El punto de linealización ocurre de forma atómica en el orden global de eventos:

- **add Exitoso:** lineariza en el `compareAndSet()` de donde el nuevo nodo se enlaza a la lista.
- **remove Exitoso:** Se lineariza exactamente en el `compareAndSet()` en el nodo pasa de estar desmarcado a marcado (*remoción lógica*).

Operaciones Inexitosas

- **add o remove Inexitosos:** Se linearizan cuando se evalúa que la condición en la clave del nodo (ya existe o no está).
- **contains(item):** depende del resultado:
 - **Exitoso:** cuando se lee la referencia del nodo coincidente y verifica que su bit de marca es `false`.
 - **Inexitoso:** cuando se lee un nodo con clave mayor, o cuando detecta que el nodo con la clave buscada está lógicamente marcado.

Algoritmo Lock-Free: Análisis de Progreso

Lock-Freedom

Garantiza **progreso global**. Si un proceso se ve obligado a reiniciar (*retry*), es porque **al menos otro proceso logró completar una operación exitosa** en el sistema.

¿Por qué los reintentos no causan Deadlock ni inanición global?

Un hilo solo reinicia el recorrido desde `head` en dos situaciones:

1. **Fallo en el CAS de limpieza (*snip*) dentro de `find()`**: Significa que otro hilo concurrente ya modificó el enlace `pred.next`. Ese otro hilo modificó con éxito la lista (ya sea insertando un nodo o removiendo físicamente otro).
2. **Fallo en el CAS de modificación (`add()` o `remove()`)**: Significa que otro hilo modificó la ventana. Hubo progreso.

Algoritmo Lock-Free: Análisis de Progreso

Progreso Inmune: `contains()`

- El método `contains()` no ejecuta instrucciones CAS ni altera punteros, por lo que nunca se bloquea ni reinicia el recorrido.
- Es **Wait-Free**: termina en un número finito de pasos determinado únicamente por la longitud física de la lista.